Dynamic Interface Power Management (PowerMAN)

Jesse Brandeburg, Brett Creeley
Intel Corporation, Ethernet Products Group
Portland, Oregon, United States
jesse.brandeburg@intel.com, brett.creeley@intel.com

Abstract

As network speeds increase, total packet buffering time is decreasing, but buffering space in hardware is not increasing at the same rate. As a result, manual system configuration and tuning must be applied to maximize performance of network interface cards, which typically comes at the cost of power consumption.

PowerMAN is a proof-of-concept to demonstrate the usefulness with an initial implementation in user space. This tool dynamically manages a network interface via existing APIs to minimize system power consumption while preventing packet drops, minimizing latency, and maintaining high throughput.

This presentation will show the results of our initial work in this space, ~~including a demo recording~~ and the analysis of power savings and other gains in resource utilization and savings.

We hope to gain input on the feasibility of managing this in-kernel or whether it should stay in user-space, as much of the work may be more appropriate for the kernel scheduler to be involved.

Keywords

Linux Kernel, Ethernet, Linux Networking, Networking Performance, Performance, Power Consumption, CPU Utilization, Throughput, Latency, Jitter, User Experience

Introduction

Since the introduction of multiple queues for networking interfaces and drivers, the choice of the number of queues allocated by default at driver load has been left to the drivers. There are varying reasons for the number of queues being limited, including hardware limits and host limits to the number of available interrupts, etc. These limits are generally set only to be maximum or best effort choices for the driver. These limits often result in several undesirable side effects including high memory usage, large latencies for transmit and receive completions (particularly when CPU power management is enabled), and, particularly if latency or dropped packets are concerned, high power utilization on the server. All these side effects are from trying to avoid issues introduced by the interaction with the network. PowerMAN is a proof-of-concept implementation that aims to dynamically manage the number of queues that are activated for default traffic.

Driver Default Configurations

Driver defaults are a useful and required mechanism, but they also cause all sorts of problems. One of the most vexing problems from a customer point of view is that every vendor selects their own defaults. In addition, we have experienced customers who need **every** workload to be supported at near ideal performance, latency, jitter, etc, with default configuration.

Transmit queues

The consensus seems to be that in an ideal case the driver should allocate one transmit queue for each available CPU, given enough hardware, memory, and interrupt resources. This provides the most efficient mode of operation when there are more transmitter threads (sockets) than there are CPUs or transmit queues, by avoiding contention on the transmit queue and qdisc lock.

Receive queues

The count and depth, of each receive queue is where the driver's default configuration and settings often result in a very large amount of memory allocated. In the case of a recent system with 112 CPUs as counted by the Linux kernel (with hyperthreading enabled), the ice.ko (for the Intel® Ethernet E800 Series Adapters and Controllers) driver allocates 32 bytes + 2kB packet buffer per receive ring entry, with 2048 ring entries per queue, and with 112 receive queues, yielding 477,102,080 bytes of memory allocated. This is for just one PF/port.

In general, most vendors with multiple receive queues enable most if not all of them to be used by the RSS (Receive Side Scaling) algorithm to select a queue for each packet. As a result, at any given moment, any receive queue (and therefore any CPU) can be selected to receive an incoming packet.

Of course, a user can modify the number of receive queues enabled by the driver using the existing 'ethtool -L ethx combined N' call. However, even with fewer queues, there might be some assigned to a CPU which is asleep.

Power Management

In general, the choice of when CPU power management is enabled is left to the user. That said, most if not all systems ship with aggressive power management enabled by default in the BIOS. Power management has some advantages, particularly on Intel platforms, where turbo can kick in to provide faster execution for workloads that are using one or a few CPUs. Of course, in saving power, less watts are used, power costs are lower, and less heat is generated by the platform. The disadvantage of power management as well as frequency management, is the CPU is not instantly available to perform driver tasks even when driven by an interrupt. This has the effect of delaying network traffic, which can cause dropped packets, queue overruns, socket buffer exhaustion, jitter, increased latency, and in general decrease performance.

Queue Management

One instance we've seen, is that a system with power management enabled, running a modern kernel, with hundreds of megabytes of receive buffer memory allocated, would drop frames for a simple netperf streaming test. This was root caused to DMA being blocked for a short amount of time, which caused our controller's internal memory to be filled. The DMA controller (uncore) was in a sleep state which blocked DMA. Figure 1. explains the limits to the on-chip buffering for different generations of Intel Ethernet Controllers, the orange line shows the 99th percentile exit latency for the DMA controller.

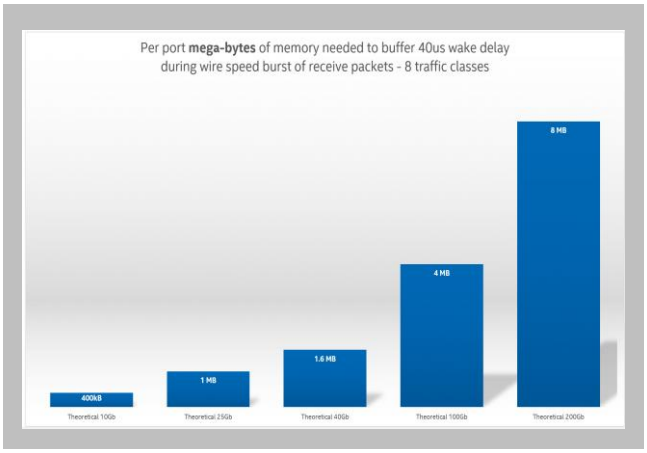


Figure 1. DMA Buffering of wire speed incoming packets, on chip, expressed in MB

In another instance, we've seen that bursts of traffic can overrun even 2048 receive buffers, given the right patterns. If a CPU isn't blocking DMA, the CPU might still be in C1, C1E, C2, C6, or some other C-state which takes time to wake from, and this impacts the driver's response time to an interrupt.

	Packet Size						
Ring Size (receive descriptor count)	64	128	256	512	1024	1518	9000
2048	13.76	24.25	45.22	87.16	171.05	251.99	1519.13

Figure 2. Packet buffering in host memory

Figure 2. shows the amount of time in micro-seconds that a 100 Gigabit connected driver, can buffer in a 2,048-entry ring (one packet per ring entry). The colors represent risk of drops with an average CPU wake latency of 40 microseconds, red on the left with 64-byte frames being highest risk, green on the right with 9,000-byte frames being the lowest risk.

Existing Solutions

The user is left in a difficult position, enable power management and suffer the latency and maybe even throughput effects due to drops, or disable power management (usually via the BIOS or system-wide at boot, or at runtime as some tuning-guides suggest), at the cost of increased power consumption, heat, and lack of turbo headroom.

Possible Heavy-Handed Administrator Actions

Turn off power management in the CPU

Poll for new packets (busy poll anyone?) – Poll on every queue?

Reduce the number of receive queues and combine with one of the above? – What about scaling up if the workload applied needs more queues?

Interrupt Moderation Settings

Interrupt moderation (both dynamic and fixed value), when combined with the sleep states from CPU power management, makes the problem even worse by delaying interrupts, often longer than the CPU needs to go **back** to sleep even after waking up recently.

Other Papers

Several other papers [1][2] have been written about RSS management, but they don't provide a simple and robust way to manage traffic without user intervention or dramatic system level changes like running alternate stacks such as DPDK, or complicated hardware changes such as eRSS [2] made.

Proposed Solution

We propose a portable, multi-vendor, software only solution initially implemented in user-space. Our solution involves using the RSS table to scale the number of active queues in real time, to the minimum needed to keep up with the offered load. We also keep the RSS targeted CPUs awake in low latency sleep states (C1) while they are waiting for packets. This has the benefit of creating a state where idle systems only have one CPU active, waiting for traffic to

arrive, as well as allowing other CPUs that might not be doing network traffic to be able to use turbo states in order to get their work completed in a shorter amount of time (more thermal headroom). Scheduling a task on a sleeping CPU should allow that CPU to update the RSS table with a new active queue. This will allow traffic to scale up without hitting delays inherent in waking a sleeping CPU.

Other alternatives not examined

Use the power management APIs of the kernel more actively from the drivers.

Experiment

Question

Can a script or program reduce the power usage of a system by reducing the number of active network queues, while also using CPU power management to keep active CPUs awake to minimize latency.

Method

A bash script was developed as a proof-of-concept. The script uses ethtool and common kernel interfaces to mange C-states and RSS indirection tables, while monitoring for dropped packets and increasing active RSS queues and CPUs as traffic increases. A static threshold for adding / removing RSS queues was established through experimentation.

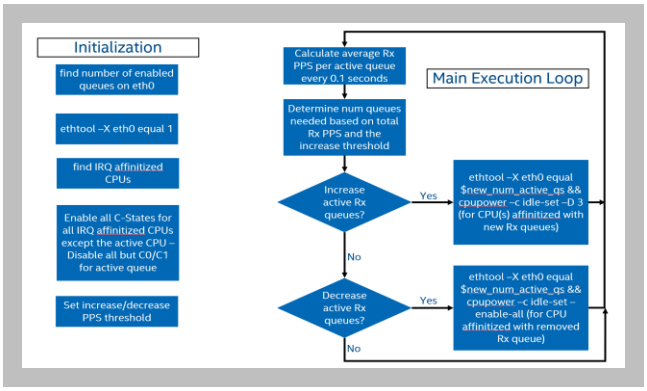


Figure 3. Script Flow

Results

We found that the PowerMAN script helped the most in reducing retransmits that were due to dropped frames. In addition, the high thread count tests benefitted the most from the activity of the PowerMAN script. Only marginal power savings were achieved by consolidating work to fewer receive queues in lower thread count tests, but drops due to deep idle states (like C6) are avoided when PowerMAN is active.

Transmit only tests (TCP) showed reduced performance because the proof-of-concept was designed to respond only to receive inputs. We confirmed that during the 128 thread transmit focused tests, there was only one receive queue active, so all the transmits and completion interrupts were likely being executed on CPUs that were frequently entering a sleep states while waiting for transmit completions.

Conclusion

PowerMAN dramatically decreases dropped frames while decreasing power usage by about 10% for higher thread count benchmarking workloads. Because of the reduced dropped frames, at the higher thread counts especially, the retransmits also dramatically decrease, which results in higher throughput and lower power utilization.

Lessons Learned and Future Methods

If we had compared our results to a system configured for minimal or no drops, and minimal latency, but setting the CPUs to C0 (poll), the difference in power savings would be much larger. Future implementations should include a focus on both transmit and receive. The transmit logic should make sure to keep actively transmitting CPUs awake above the minimum latency requirement. Other opportunities for further testing include results from XDP, AF_XDP, packet generators, forwarding workloads, UDP tests and including busy-poll for active queues.

Future

Vision

The kernel scheduler will actively manage the active queues of the interface to direct traffic to the CPU(s) it chooses. It will get feedback from the threaded NAPI handler, CPU utilization, and per-queue or per-CPU counters about how many packets per second and drops are occurring so it can scale queues for receive traffic. The ethtool interface to RSS control is the most likely way for the scheduler to control this, but the other feedback mechanisms need to be formalized so that the scheduler can get the right inputs.

Possible Small Steps

Scheduler becomes aware of:

- Cycles spent in a NAPI handler
- What queues are part of which netdev
- Queue overruns (drops) or full queues

Scheduler then integrates data of cycles used and packet drops, to choose when to enable more RSS queues to balance the workload.

Acknowledgements

The authors would like to thank our employer for supporting pathfinding and employee growth by allowing us to work on this project.

References

RSS++: load and state-aware receive side scaling
[1] Tom Barbette, Georgios P. Katsikas, Gerald Q. Maguire, and Dejan Kostić. 2019. "RSS++: load and state-aware receive side scaling." *In Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies (CoNEXT '19). Association for Computing Machinery, New York, NY, USA,* 318–333. DOI: https://doi.org/10.1145/3359989.3365412

Elastic RSS: Co-Scheduling Packets and Cores Using Programmable NICs
[2] Alexander Rucker, Muhammad Shahbaz, Tushar Swamy, and Kunle Olukotun. 2019. "Elastic RSS: Co-Scheduling Packets and Cores Using Programmable NICs." *In Proceedings of the 3rd Asia-Pacific Workshop on Networking 2019 (APNet '19). Association for Computing Machinery, New York, NY, USA, 71–77.* DOI: https://doi.org/10.1145/3343180.3343184

Author Biographies

Jesse Brandeburg is a Principal Software Engineer at Intel, with 28 years of experience as a software developer. He joined Intel in 1994 after graduating from the University of Portland with a degree in Electrical Engineering. Since 2000, he has developed Linux drivers for Intel Ethernet adapters.

Brett Creeley is a Software Engineer at Intel, with a little over 3 years of experience. He graduated from Portland State University with a bachelor's degree in electrical and computer engineering. His primary focus is development of Linux drivers for Intel Ethernet adapters.